

**МИНОБРНАУКИ РОССИИ**  
**САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ**  
**ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ**  
**«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)**  
**Кафедра математического обеспечения и применения ЭВМ**

**ОТЧЕТ**  
**по лабораторной работе №4**  
**по дисциплине «Построение и анализ алгоритмов»**  
**Тема: Поиск подстроки в строке**

Студент гр. 4345

Кравцов С. А.

Преподаватель

Жангиров Т.Р.

Санкт-Петербург

2026

## **Цель работы**

Изучить и реализовать алгоритм Кнут-Моррис-Пратт.

### **Постановка задачи**

Часть 1. Реализуйте алгоритм КМП и с его помощью для заданных шаблона  $P$  ( $|P| \leq 25000$ ) и текста  $T$  ( $|T| \leq 500000$ ) найдите все вхождения  $P$  в  $T$ .

*Ввод:* строка  $P$  (шаблон), строка  $T$  (текст).

*Вывод:* индексы начал вхождений  $P$  в  $T$ , разделенные запятой; если  $P$  не входит в  $T$ , то вывести -1.

#### **Ввод**

ab

abab

#### **Вывод**

0, 2

Часть 2. Заданы две строки  $A$  ( $|A| \leq 5000000$ ) и  $B$  ( $|B| \leq 5000000$ ). Определить, является ли  $A$  циклическим сдвигом  $B$  (это значит, что  $A$  и  $B$  имеют одинаковую длину и  $A$  состоит из суффикса  $B$ , склеенного с префиксом  $B$ ). Например, defabc является циклическим сдвигом abcdef.

*Вход:* первая строка  $A$ , вторая строка  $B$

*Выход:* если  $A$  является циклическим сдвигом  $B$ , то индекс начала строки  $B$  в  $A$ ; иначе -1. Если возможно несколько сдвигов, вывести первый индекс.

#### **Ввод**

defabc

abcdef

#### **Вывод**

3

## Выполнение работы

При наивном решении задачи поиска подстроки в строке очевидным кажется решение имеющее сложность  $O(n*m)$ , где  $n$  – длина текста,  $m$  – длина шаблона. Такая сложность обусловлена тем, что для каждого символа от 1-ого до  $(n-m+1)$ -ого производятся сравнения всех последующих  $m$  символов с шаблоном. Это неоптимально, так как наивный алгоритм "забывает" результаты предыдущих сравнений. Префикс-функция позволяет оптимизировать этот процесс: она анализирует структуру самого шаблона и выявляет в нем повторяющиеся части. Благодаря этому, при несовпадении символов, мы точно знаем, на какую позицию в шаблоне можно безопасно откатиться, не возвращаясь назад по тексту. Это исключает заведомо ложные проверки и позволяет достичь линейной сложности  $O(n + m)$ .

Были реализованы методы `kmp` и `shift_kmp`. Функции имеют почти аналогичную логику.

1. Префикс функция - процесс начинается со второго символа, так как для одного символа значение всегда равно 0. Основной цикл `for` последовательно перебирает символы шаблона, пытаясь увеличить текущую длину совпадения  $j$ . Если следующий символ  $p[i]$  не совпадает с ожидаемым символом префикса  $p[j]$ , срабатывает цикл `while`, который не сбрасывает поиск в ноль, а оперативно откатывает  $j$  к значению  $pi[j-1]$ . Это позволяет проверить наличие более коротких повторяющихся цепочек внутри уже обработанной части, не совершая лишних сравнений. Когда совпадение найдено или достигнут счётчик 0, итоговое значение  $j$  записывается в массив  $pi$ , фиксируя инструкцию по прыжкам для основного алгоритма поиска. Благодаря тому, что  $j$  увеличивается максимум на единицу за итерацию, суммарное количество откатов в `while` также ограничено, что обеспечивает линейную сложность работы  $O(m)$ .

2. Алгоритм Кнута-Морриса-Пратта - процесс поиска проходит по тексту  $t$  линейно: указатель  $i$  движется только вперед от начала до конца, никогда не возвращаясь назад. Параллельно используется указатель  $q$ ,

который отслеживает количество совпавших символов шаблона в текущей позиции. Если очередной символ текста совпадает с ожидаемым символом шаблона  $p[q]$ , счетчик  $q$  увеличивается, и алгоритм переходит к следующему шагу. В случае разрыва совпадения в действие вступает цикл `while`, который откатывает  $q$  к значению  $pi[q-1]$ . Это перемещает шаблон относительно текста так, что уже проверенный хвост совпадения становится новым началом, экономя время на повторных проверках. Когда счетчик  $q$  становится равен длине шаблона  $m$ , алгоритм фиксирует вхождение и вычисляет его индекс. Для продолжения поиска (с целью найти перекрывающиеся вхождения)  $q$  снова откатывается по значению префикс-функции, подготавливая систему к следующему шагу.

`shift_kmp` отличается от `kmp` лишь тем, что поиск производится в виртуально удвоенной строке, для проверки сдвига, а также вхождение единственно, поэтому результат единственен.

Сложность алгоритма по времени  $O(N+M)$ , где  $M$  – длина текста,  $N$  – длина шаблона. Обусловлена тем, что префикс функция задается циклом за  $O(M)$ , а алгоритм поиска единожды проходит по заданному тексту, а цикл возврата к предыдущим  $q$  константно ограничен  $\Rightarrow O(N)$ .

Сложность алгоритма по памяти  $O(M+N)$ , где  $M$  – длина текста,  $N$  – длина шаблона. Обусловлена хранением строк.

Исходный код программы см. в приложении А.

Результаты тестирования см. в Таблице 1-2.

Таблица 1 – Тестирование `kmp`

| № | Входные данные         | Выходные данные | Комментарий                    |
|---|------------------------|-----------------|--------------------------------|
| 1 | aba<br>ababa           | 0,2             | Перекрытие                     |
| 2 | aa<br>aaabaaaa         | 0,1,4,5,6       | Избыточность                   |
| 3 | abacaba<br>abacabacaba | 0,4             | Проверка откатов $q = pi[q-1]$ |
| 4 | asd<br>zxczxcvbn       | -1              | Отсутствие вхождения           |

Таблица 2 – Тестирование shift\_kmp

| № | Входные данные   | Выходные данные | Комментарий                     |
|---|------------------|-----------------|---------------------------------|
| 1 | aaaaa<br>aaaaa   | 0               | Максимальная<br>симметрия       |
| 2 | asddsa<br>dsaasd | 3               | Стандартный случай              |
| 3 | ababaa<br>baaaba | 3               | Проверка откатов $q = p_i[q-1]$ |
| 4 | asdasd<br>asdzx  | -1              | Разная длина                    |

## **Выводы**

В ходе работы был реализован строковый поиск на базе алгоритма КМП с линейными характеристиками сложности  $O(P + T)$ . Функционал включает поиск всех вхождений паттерна в текст и механизм определения циклической эквивалентности строк. Алгоритм оптимизирован по памяти и времени, обеспечивая эффективную обработку данных без избыточных сравнений.

## ПРИЛОЖЕНИЕ А

### ИСХОДНЫЙ КОД ПРОГРАММЫ

Файл kmp.py

```
def kmp(p, t):
    m = len(p)
    n = len(t)

    if m == 0:
        return

    print(f"\nВычисляем префикс-функцию для шаблона '{p}'")
    pi = [0] * m
    j = 0
    for i in range(1, m):
        while j > 0 and p[i] != p[j]:
            j = pi[j-1]
        if p[i] == p[j]:
            j += 1
        pi[i] = j

    print(f"pi-массив = {pi}")

    print(f"Начинаем поиск в тексте длиной {n} символов")
    res = []
    q = 0
    for i in range(n):
        print(f"Шаг {i}: Текущий символ текста '{t[i]}', пытаемся сопоставить с '{p[q]}' (q={q})")
        while q > 0 and p[q] != t[i]:
            q = pi[q-1]
        print(f"Символы не совпали, откат по префикс функции: q = {q}")

        if p[q] == t[i]:
            q += 1
        print(f"Символы совпали идем вперед по префикс-функции: q = {q}")
```

```

        if q == m:
            start_index = i - m + 1
            print(f"    Найдено вхождение, индекс начала: {start_index}")
            res.append(str(start_index))
            q = pi[q-1]
            print(f"    Сдвигаемся для поиска перекрытий, новый q = {q}")

    return res

res = kmp(input(), input())

if not res:
    print("-1")
else:
    print("Индексы вхождений:", ", ".join(res))

Файл shift_kmp.py

def shift_kmp(a, b):
    n = len(a)
    m = len(b)

    if n != m:
        print("-1")
        return
    if n == 0:
        print("0")
        return

    print(f"\nВычисляем префикс-функцию для шаблона '{b}'")
    pi = [0] * m
    j = 0
    for i in range(1, m):
        while j > 0 and b[i] != b[j]:
            j = pi[j-1]
        if b[i] == b[j]:
            j += 1
        pi[i] = j

```

```

print(f"pi-массив = {pi}")

q = 0
for i in range(2 * n):
    print(f"Шаг {i}: Символ текста A[{i % n}] = '{a[i % n]}',
сопоставляем с B[{q}] = '{b[q]}'")

    while q > 0 and a[i % n] != b[q]:
        q = pi[q-1]
        print(f"    Символы не совпали, откат по префикс функции: q
= {q}")

    if a[i % n] == b[q]:
        q += 1
        print(f"    Символы совпали идем вперед по префикс-функции:
q = {q}")

    if q == m:
        print(f"    Найдено вхождение, Индекс начала: {i - m + 1}")
        print(i - m + 1)
        return

print("-1")

shift_kmp(input(), input())

```